

Deployment

Para orquestar los contenedores, podríamos pasar de Docker Compose a Kubernetes (o ECS). También tiene sentido añadir un API Gateway (AWS API Gateway) como capa de entrada: De ahí exponemos un único endpoint, enrutamos las llamadas, y podemos aplicar autenticación, rate-limiting y WAF sin tocar cada servicio. Para servir la API, se puede combinar Uvicorn/Gunicorn con Nginx con HTTPS en un load balancer. En cuanto a secretos y variables de entorno si se toma una arquitectura basada en AWS se gestionan idealmente con AWS Secrets Manager, Vault u otro servicio similar.

Base de datos

En lugar de un container local, sería ideal usar un servicio gestionado (RDS, Cloud SQL) que ofrezca backups y escalado.

Capa de caché

Para aligerar las búsquedas semánticas en PGVector, se podría introducir Redis y almacenar ahí resultados frecuentes con TTL para no ir a buscar a la DB consultas ya repetidas por los usuarios.

Procesamiento asíncrono para el sumador

Para no bloquear al usuario en cada petición, podríamos implementar una cola de mensajes (RabbitMQ, AWS SQS) y disparar el sumador:

- Después de N mensajes de una conversación, se envía un job a la cola.
- Worker dedicado procesa los jobs y actualiza el resumen en segundo plano.

Esto también podría hacerse con una alternativa “más barata” usando BackgroundTasks de fastAPI.

Debounce para los mensajes

Sugiero agregar un debounce con un delta de tiempo de esperar (por ejemplo 2 segundos) tras el último mensaje recibido.

- Reiniciar el temporizador si llega un nuevo mensaje dentro de ese período.
- Solo cuando pasen esos N segundos sin actividad, ejecutar el ciclo del agente

Así agrupamos varias interacciones en una sola llamada, reducimos carga en el modelo y mantenemos la respuesta al usuario rápida, sin bloquear la request original.

Autenticación y límites

Para proteger los endpoints, podríamos implementar OAuth2/JWT y definir roles o scopes. Y para evitar abusos, convendría aplicar rate-limiting (por ejemplo con SlowAPI, el API Gateway o una solución propia en Redis).

Logging

Agregaría una capa más de logging. Centralizar el logging en un archivo de configuración y trackearia tiempos de respuestas, uso de tokens, uso de la API, rate limits.

Observabilidad

Sería útil exponer métricas con herramientas como Prometheus (latencias, errores) y visualizarlas en Grafana. Los logs podrían centralizarse en ELK Stack o un servicio en la nube, y para trazas distribuidas podríamos usar OpenTelemetry.

CI/CD y despliegues

Podríamos montar pipelines en GitHub Actions o GitLab CI que construyan, prueben y desplieguen en cada cambio. Incluyamos tests unitarios e integración, y definamos un plan de rollback que sea rápido de ejecutar basado en casos de testeos (como implemente en evaluator) pero versionados.